



Remote Control Vehicle Project

EE325- Embedded Systems

Professor: Dr. Tarik Namas

Student: Farah Mašić, 230302245

Merima Prcanović, 230302264

Davud Sadikaj, 230302239

Sarajevo, Spring , 2026

Table of Contents

Table of figures	3
1. Introduction	5
2. Materials and Components Used	6
3. System Design and Methodology	8
4. Code Explanation	11
4.1 DC Motors	11
4.1.1 Overview	11
4.1.2 Motor control logic.....	12
4.1.3 Timer and register control	15
4.2 Servo Motors	16
4.2.1 Overview	16
4.2.2 Servo angle logic.....	17
4.2.3 Servo sweep	18
4.3 USART Communication	19
4.3.1 Overview	19
4.3.2 Packet Protocol Design	20
4.3.3 Role of the ESP32	21
4.3.4 STM32- UART Reception and Vehicle Control	23
4.4 Final Integrated Code.....	24
4.4.1 Overview	24
4.4.2 Limit definitions	25
4.4.3 Motor functions.....	27
4.4.4 Steering functions	28
4.4.5 Processing and RC initialization	30
5. Incremental Results and Final Results	33
5.1 Overview	33
5.2 DC motors	33
5.3 Servo motor	33
5.4 Setup using Buck converter	34
5.5 Final setup.....	36
6. Challenges and Troubleshooting	37
6.1 Challenges with the L298N Motor Driver	37
6.2 Challenges in Soldering and Wiring.....	37
6.3 Challenges with the Servo Motor	38
6.4 Challenges Caused by Timer and UART Conflicts.....	38
6.5 Challenge Related to Current Supply	38
7. Conclusion	39
References	40

Table of figures

Figure 1 Materials and components used	7
Figure 2 3D printed chassis	7
Figure 3 Initial schematic	8
Figure 4 Final schematic	10
Figure 5 DC motor MX configuration	11
Figure 6 Initial DC motor constraints	12
Figure 7 Motor 1 functions	13
Figure 8 Motor 2 functions	14
Figure 9 Loop controlling forward movement	14
Figure 10 Loop controlling backward movement	15
Figure 11 Functions for setting PWM frequency	15
Figure 12 Reset PWM signal.....	16
Figure 13 Servo motor MX configuration	17
Figure 14 Servo angle function.....	17
Figure 15 Starting PWM signal	18
Figure 16 Servo sweep loop	18
Figure 17 Timer configuration for servo PWM	19
Figure 18 ESP32 code snippet for sending the data packet to STM32	20
Figure 19 Libraries included	21
Figure 20 Hardware Peripheral Initialization	22
Figure 21 Main Loop	22
Figure 22 Dabble joystick gamepad.....	23
Figure 23 UART Initialization	23
Figure 24 Packet Parser	24
Figure 25 Final MX configuration	25
Figure 26 Servo limits definition	26
Figure 27 Motor driver input pins.....	26
Figure 28 USART2 definitions	27
Figure 29 Function for stopping motors	27
Figure 30 Function for forward rotation of motors	27
Figure 31 Function for backward rotation of motors.....	28
Figure 32 Applying throttle from joystick value	28
Figure 33 Steering control	29
Figure 34 Emergency stop	29
Figure 35 Packet processing	30
Figure 36 RC initialization	30
Figure 37 Main RC processing loop	31
Figure 38 Main function.....	32
Figure 39 DC motors connected to motor driver	33

Figure 40 Servo Motor rotation	34
Figure 41 Servo motor rotation.....	34
Figure 42 Connected components including Buck converter	35
Figure 43 Components placed on chassis	35
Figure 44 Final RC vehicle setup	36
Figure 45 RC vehicle project at the Makers Faire 2026.....	39
Table 1 Pinout configuration for STM32 and ESP32	20
Table 2 The actual meaning of the bytes transmitted over the channel ESP32 -> STM32	
.....	21

1. Introduction

A remote-controlled vehicle is a common embedded systems project because it combines software programming with physical hardware control. In this project, the vehicle is designed to receive wireless commands and respond through movement and steering. This allows several important embedded systems concepts to be demonstrated in one practical system, including GPIO control, timer configuration, Pulse Width Modulation, clock setup, and USART2 communication.

The main objective of this project was to show how a microcontroller can be used to control external hardware components. The DC motors are responsible for the movement of the vehicle, while the servo motor is used for steering. Bluetooth communication allows the user to send commands wirelessly, which are then processed by the microcontroller and converted into motor actions.

This project also highlights the importance of hardware and software working together correctly. The final behavior of the vehicle depends not only on the program logic, but also on accurate wiring, stable power supply, correct timer settings, and reliable communication between modules. Because of this, the project provides a realistic example of the challenges involved in embedded systems development.

2. Materials and Components Used

The implementation of the remote-controlled vehicle required both electronic components and software development tools. Since the project focuses on embedded systems, each component was selected to demonstrate a specific microcontroller feature, such as GPIO control, PWM signal generation, timer configuration, and serial communication. The list is as follows:

1. DC Motors x2
2. Motor Driver L298N
3. Servo Motor
4. ESP32
5. STM32 microcontroller
6. Buck converter (step- down converter)
7. 9V battery
8. Breadboard
9. Powerbank
10. USB mini B cable
11. USB-C cable
12. Capacitor 100uF x2
13. Wires
14. Battery snap for 9V battery
15. Wheels x4
16. Switch
17. Copper wire
18. 3D printed Chassis
19. Screwdriver
20. STM32IDE Software
21. STM32MX Software

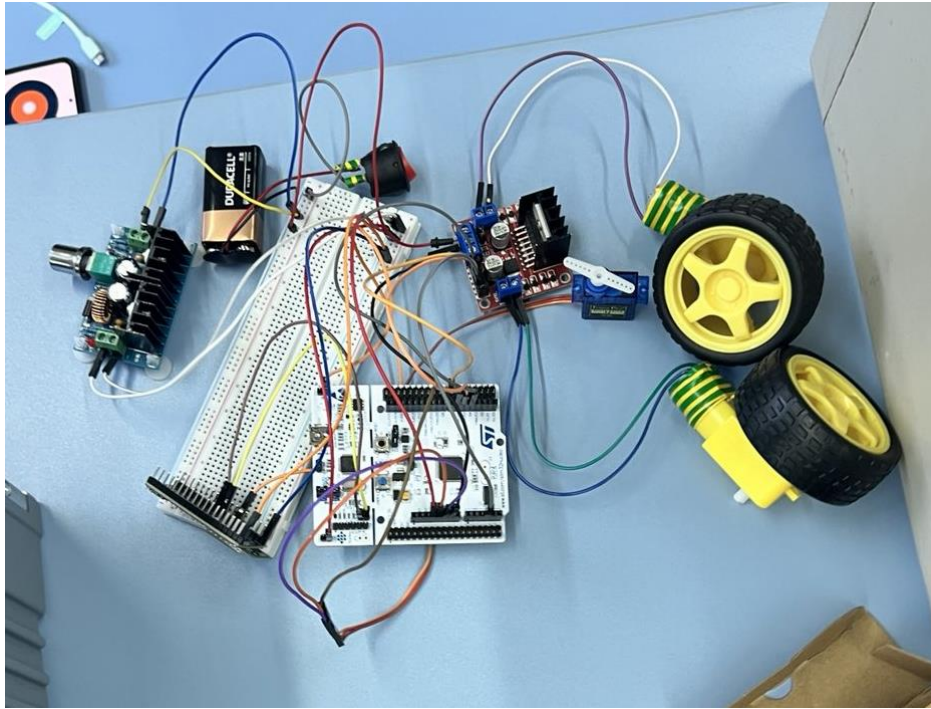


Figure 1 Materials and components used



Figure 2 3D printed chassis

3. System Design and Methodology

Going into this project, it was clear that the system would require interaction between several main parts: the user controlling the vehicle, the ESP32 communication module, the STM32 microcontroller, and the motor control hardware. With the help of industry professionals and online tutorials, the components were categorized based on their required voltage and current needs. This was important because not all components in the circuit operate at the same voltage level.

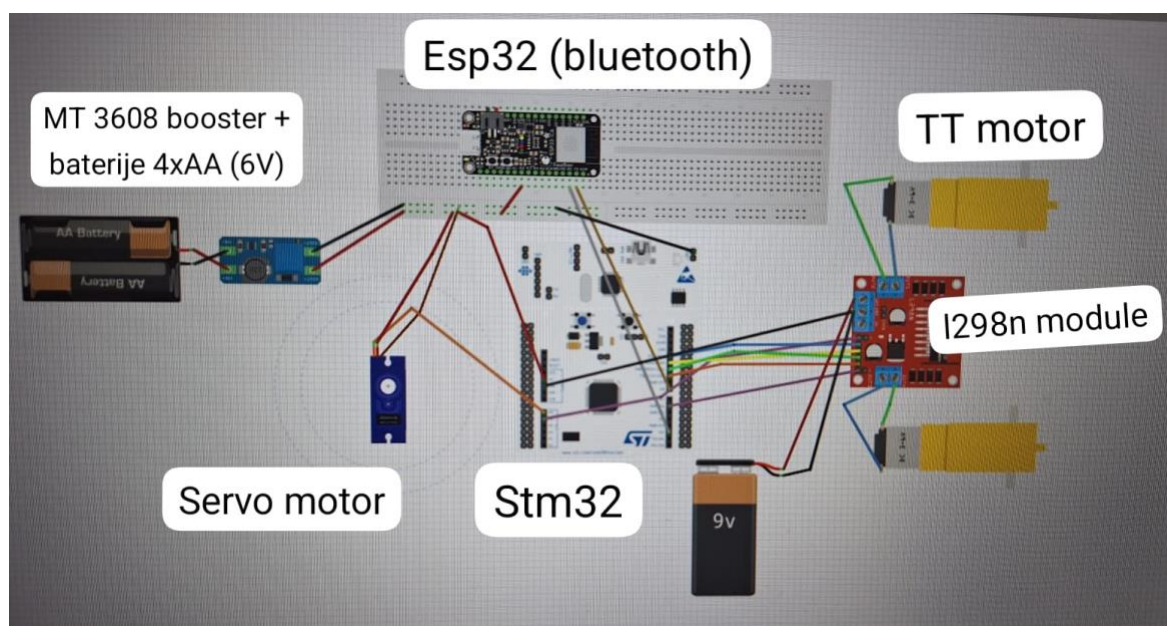


Figure 3 Initial schematic

As mentioned in the materials list, a 9V battery was used as the main voltage source for the circuit. However, most of the electronic components require lower voltage levels, usually between 3.3V and 5V. To solve this issue, a buck converter, also known as a step-down converter, was used. The buck converter connects to the main voltage supply and reduces the voltage to a suitable level for components that require lower voltage, such as 3.3V. This prevents components from being damaged by excessive voltage and helps the system operate more reliably.

The motor driver is connected directly to the 9V battery because the DC motors require a higher voltage to operate properly. Since the STM32 microcontroller cannot provide enough current to drive the DC motors directly, the motor driver acts as an interface between the

microcontroller and the motors. The DC motors are connected to the motor driver outputs, while the input pins of the motor driver are connected to specific GPIO pins on the STM32 board. These GPIO pins control the forward and backward movement of each motor by changing their HIGH and LOW states.

The servo motor is used for steering control and requires a Pulse Width Modulation signal. PWM is a signal that repeatedly switches between ON and OFF states. The position of the servo motor depends on the width of the HIGH part of the signal. For example, a shorter pulse turns the servo in one direction, a middle pulse centers it, and a longer pulse turns it in the opposite direction. This makes PWM suitable for controlling the steering angle of the vehicle.

Timers are used to generate the PWM signal accurately. The STM32 timer counts based on the configured clock frequency and creates a repeating signal with a specific period and duty cycle. For a servo motor, timing must be stable because even small changes in pulse width can affect the servo angle. In this project, the timer is configured to create the PWM signal needed to move the servo left, right, or back to the center position. The servo motor is connected to the STM32 through its signal pin, while its power and ground pins are connected to the appropriate voltage and ground lines.

Finally, the ESP32 is responsible for the wireless control part of the project. Instead of the STM32 receiving commands directly from a wired input, the ESP32 allows the vehicle to be controlled remotely. Once the ESP32 receives a command from the user, it sends the command data to the STM32 using serial communication. On the STM32 side, USART2 is used to receive this data. USART, or Universal Synchronous/Asynchronous Receiver Transmitter, is a serial communication interface that allows data to be transferred between devices using transmit and receive lines.

This communication is made possible through the TX and RX pins. The TX pin of the ESP32 is connected to the RX pin of the STM32, while the RX pin of the ESP32 is connected to the TX pin of the STM32. This crossed connection allows one device to transmit data while the other receives it. The ESP32 is also connected to the breadboard power rail receiving 3.3V, which matches its operating voltage. Through this connection, the STM32 can read commands from the ESP32 and use them inside the program logic to control the motors and steering.

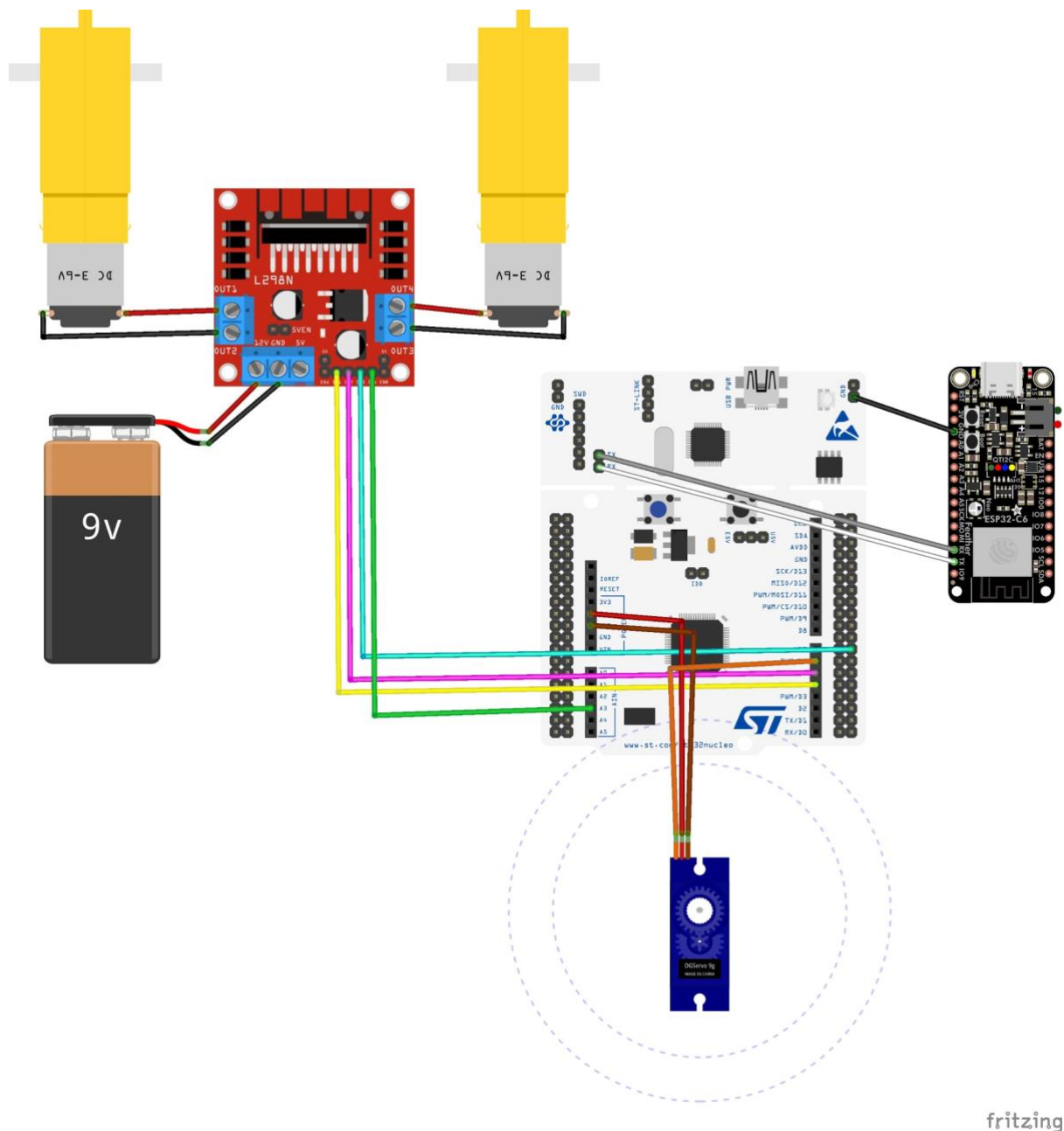


Figure 4 Final schematic

By removing the battery pack and voltage regulator, subbing them out with a powerbank both the STM32 microcontroller and ESP32 module got a constant surge of voltage and current, enough to power both consistently.

4. Code Explanation

4.1 DC Motors

4.1.1 Overview

As the project consists of multiple hardware components, all with their own specifics we did the work incrementally. The reason for this was to ensure the components worked and the logic was sound, as we completed more tasks on the checklist the codes got integrated and finally we got our working code and final product. We first started with DC motors which were initially written using direct register programming. We treated the DC motors like we did Servo motor later, so using PWM signals. One of the reasons for this was also the fact that the ESP32 module was implemented later, so DC motors were code controlled. To show that they worked we timed forward and backward rotation. The MX configuration for this was:

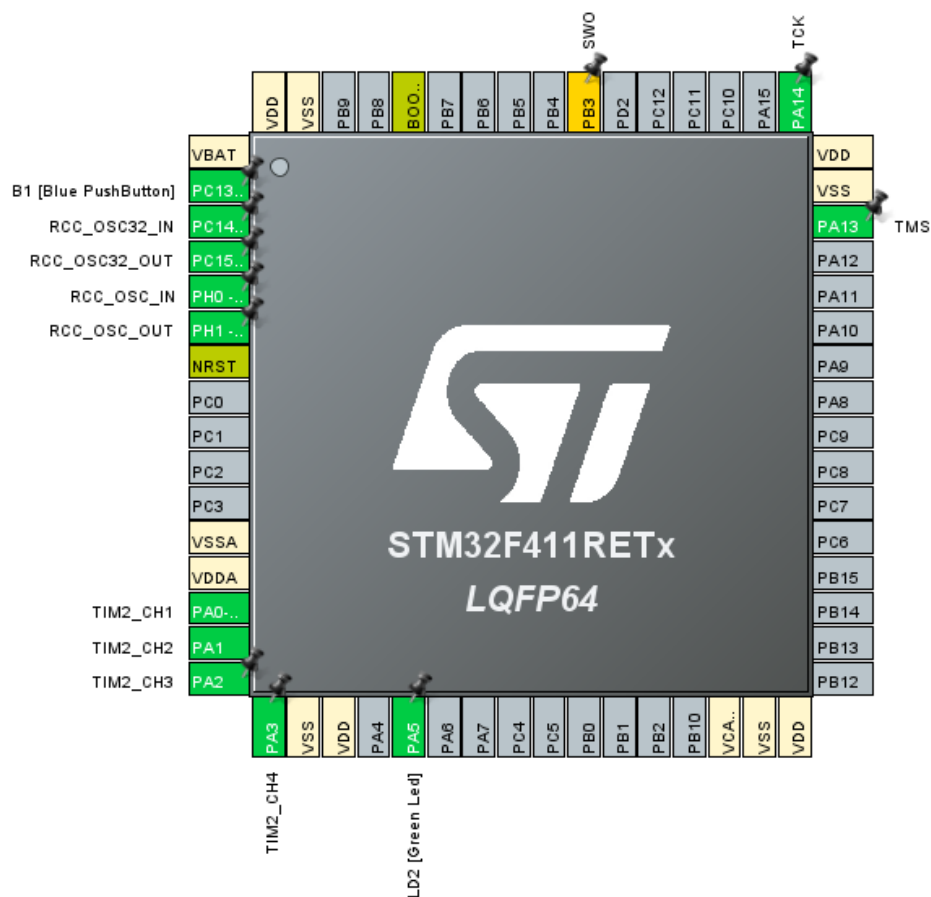


Figure 5 DC motor MX configuration

As seen in the figure above we used TIM2, or timer 2 whose 4 channels correlate to pins PA0-3. For Motor 1, pins PA0 and PA1 were connected to IN1 and IN2, while for Motor 2, initially, pins PA2 and PA3 were connected to IN1 and IN2. Once troubleshooting and realising Motor 2 was blocked because of USART2 logic being on pins PA2 and PA3, we switched the pins to PB6 and PB7 which got us a satisfactory result.

```
40  
41 #define PWM_MAX          255  
42 #define STEP_DELAY_MS    5  
43 #define PAUSE_MS         1000  
44  
.. ..
```

Figure 6 Initial DC motor constraints

These constraints we define which directly affect the rotation speed and delay intervals of the motors. PWM_MAX defines highest PWM value of 255, meaning PWM signal can be from 0-255 which matches the timer auto-reload register. At 0, the motors receive no power meaning they will not rotate, while at 255 the motors will run at maximum speed. STEP_DELAY_MS sets the delay between each increase in PWM value when the motor speed is gradually increased. This creates a smooth acceleration instead of an immediate jump to full speed. PAUSE_MS defines how long the program waits after the motors are stopped before continuing to the next operation.

4.1.2 Motor control logic

```

126
127 void Motor1_Forward(uint8_t speed)
128 {
129     TIM2->CCR1 = speed;
130     TIM2->CCR2 = 0;
131 }
132
133 void Motor1_Reverse(uint8_t speed)
134 {
135     TIM2->CCR1 = 0;
136     TIM2->CCR2 = speed;
137 }
138
139 void Motor1_Stop(void)
140 {
141     TIM2->CCR1 = 0;
142     TIM2->CCR2 = 0;
143 }
144

```

Figure 7 Motor 1 functions

As mentioned, in this code direct register programming was used. The same is the case in the functions seen in the figures above. The CCR (capture/ compare register) is associated with a certain timer channel. In the code above, CCR1 is associated with Timer 2 Channel 1 which we set to rotate for forward movement, while it is set to 0 when wanting to reverse. CCR2 is associated with Timer 2 Channel 2 which is set to a desired speed once backwards rotation is wanted.

```

144
145 void Motor2_Forward(uint8_t speed)
146 {
147     TIM4->CCR1 = speed;
148     TIM4->CCR2 = 0;
149 }
150
151 void Motor2_Reverse(uint8_t speed)
152 {
153     TIM4->CCR1 = 0;
154     TIM4->CCR2 = speed;
155 }
156
157 void Motor2_Stop(void)
158 {
159     TIM4->CCR1 = 0;
160     TIM4->CCR2 = 0;
161 }
162

```

Figure 8 Motor 2 functions

Once configured for PB6 and PB7, Motor 2 had to be configured on Timer 4. In this case logic is the same as previous PB6 which corresponds to Channel 1 gains desired speed for forward movement, and is set to 0 for backward rotation and vice versa for PB7.

```

for (int i = 0; i <= PWM_MAX; i++)
{
    Motor1_Forward(i);
    Motor2_Forward(i);

    delay_ms(STEP_DELAY_MS);
}

```

Figure 9 Loop controlling forward movement

This loop gradually increases the speed of both motors in the forward direction. The variable `i` starts from 0 and increases until it reaches the maximum PWM value. As the PWM duty cycle increases, the motors receive more power and their speed increases gradually instead of instantly switching to full speed.

```
for (int i = 0; i <= PWM_MAX; i++)
{
    Motor1_Reverse(i);
    Motor2_Reverse(i);

    delay_ms(STEP_DELAY_MS);
}
```

Figure 10 Loop controlling backward movement

This loop performs the same speed process, but in the reverse direction. Both motors slowly increase their reverse speed from 0 to the maximum PWM value.

4.1.3 Timer and register control

```
/* 16MHz / 16 = 1MHz */
TIM2->PSC = 15;

/* 1MHz / 256 = 3.9kHz PWM */
TIM2->ARR = PWM_MAX;
```

Figure 11 Functions for setting PWM frequency

These lines configure the PWM frequency for Timer 2. The prescaler divides the 16 MHz system clock by 16, resulting in a timer clock of 1 MHz. Since the auto-reload register is set to 255, the PWM frequency is approximately 3.9 kHz. This frequency is suitable for basic DC motor control.

```
TIM2->CCR1 = 0;  
TIM2->CCR2 = 0;
```

Figure 12 Reset PWM signal

Finally the CCR is set to 0, ensuring that PWM is completely disabled on each channel respectively. This prevents the motor from receiving unintentional signals, avoiding scenarios where both forward and backward signals are active at the same time.

This code proved effective for the first trial and helped us get used to these components and their logic, once ESP32 module is introduced the DC motor logic does not stay the same, but troubleshooting the correct pins on STM32 helped in the final code resolution.

4.2 Servo Motors

4.2.1 Overview

The next component to be configured was the servo motor. We decided to implement this instead of having four DC motors because it allows left-right movement. For the first testing of this component, we configured a timer, specifically Timer2. The timer is used to create the PWM signal needed to control the servo motor. The servo needs a signal that repeats every 20 ms. Inside that signal, the length of the HIGH pulse decides the servo angle. By changing the compare value of the timer, the code changes the HIGH time of the PWM signal from around 1 ms to 2 ms, which corresponds to servo angles from 0° to 180° which controls the servo position accurately. Timer2 channel 1 is configured for the input of the servo, which also has a ground and 5V output pin that needs to be connected to STM32. This gives us a configuration that looks like this:

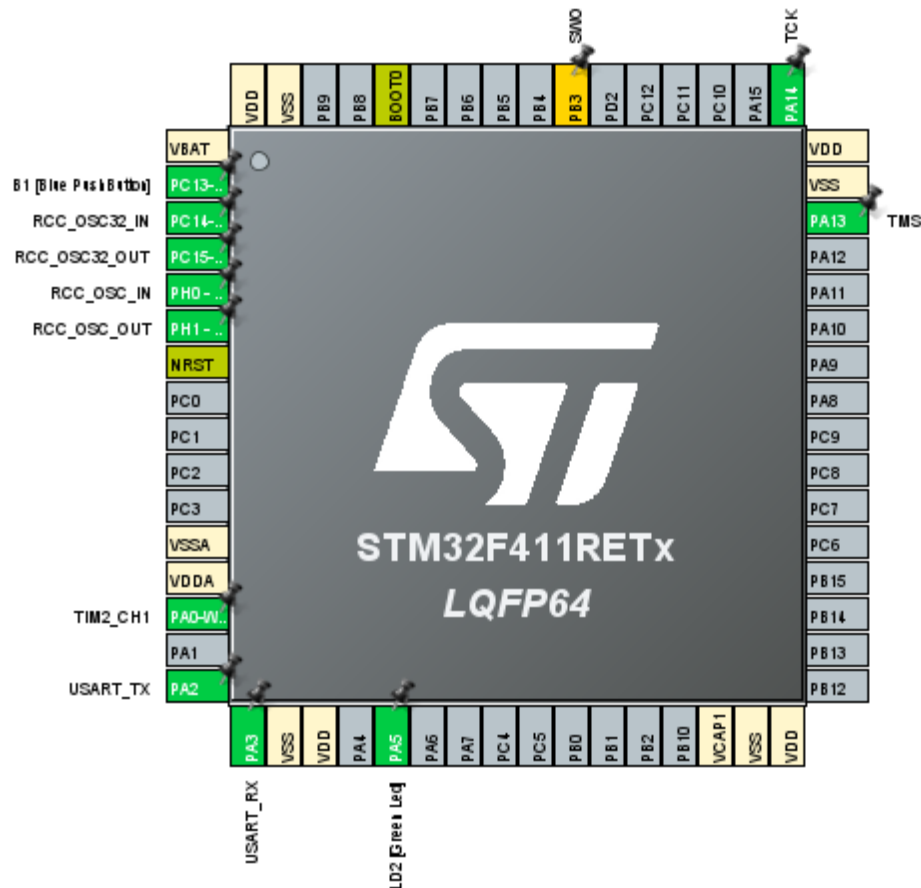


Figure 13 Servo motor MX configuration

4.2.2 Servo angle logic

```

38 void Servo_SetAngle(uint8_t angle)
39 {
40     if (angle > 180) angle = 180;
41     uint32_t pulse = 1000u + ((uint32_t)angle * 1000u / 180u);
42     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
43 }

```

Figure 14 Servo angle function

This function takes the received desired servo angle and checks if its in the range of 0-180°. After checking, the first line limits the angle to be 180°. This prevents invalid inputs sent to the servo. Pulse width is calculated using the formula:

$$1000 + \frac{angle * 1000}{180}$$

This converts the angle into a PWM pulse width that is between 1-2 ms. Each pulse width corresponds to a servo angle, which in turn affects servo position. The final line changes the compare register of TIM2 Channel 1, which changes the position of the servo motor.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim2);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

Figure 15 Starting PWM signal

These lines start the timer and enable PWM output on TIM2 Channel 1. The base timer must run so that the counter can count continuously, while the PWM start function activates the PWM signal on the selected timer channel.

4.2.3 Servo sweep

```
/* Sweep 0° → 180° */
for (uint8_t angle = 0; angle <= 180; angle += 10)
{
    Servo_SetAngle(angle);
}

/* Sweep 180° → 0° */
for (int16_t angle = 180; angle >= 0; angle -= 10)
{
    Servo_SetAngle((uint8_t)angle);
}
```

Figure 16 Servo sweep loop

These loops make the servo move from 0 degrees to 180 degrees and then back from 180 degrees to 0 degrees. The angle changes in steps of 10 degrees. Each time the loop runs, the `Servo_SetAngle()` function calculates a new PWM pulse width and sends it to the servo.

```
htim2.Init.Prescaler      = 83;  
htim2.Init.CounterMode   = TIM_COUNTERMODE_UP;  
htim2.Init.Period        = 19999;
```

Figure 17 Timer configuration for servo PWM

These two values define the PWM frequency. The system clock is 84 MHz. By using a prescaler of 83, the timer clock becomes 1 MHz, meaning each timer tick is equal to 1 μ s. The period is set to 19999, so the timer counts 20,000 ticks which is equal to 20ms, which corresponds to 50 Hz.

4.3 USART Communication

4.3.1 Overview

USART stands for universal asynchronous receiver/transmitter, a very simple protocol of transferring data by sending the bits one by one through two wires. In the system, there are two types of controllers working together: ESP32 DevKit V1 (classic) that takes care of the Bluetooth Low Energy (BLE) communications with the Dabble app on the user's phone, and STM32F411RE, the main controller of the vehicle that takes control of motors. The channel through which the two controllers communicate with each other is established through UART. Knowing about this communication channel is fundamental because all of the control instructions come from the user's phone, sent wirelessly to ESP32 and further through UART to the STM32.

In this project, USART2 on the STM32 is configured to communicate with the ESP32's UART2 peripheral at 115200 baud, 8 data bits, no parity, and 1 stop bit (commonly written as 115200 8N1). The physical connection requires only two signal lines plus a shared ground.

ESP32 PIN	STM32 PIN	FUNCTION
GPIO 17 (TX)	RX pin on the ST Link	Data travels from ESP32 to STM32
GPIO 5 (RX)	TX pin on the ST Link	Data travels from STM32 to ESP32
GND	GND	Common ground between two MCU-s

Table 1 Pinout configuration for STM32 and ESP32

4.3.2 Packet Protocol Design

The UART transmission is byte-wise without any structure at all. In order to facilitate easy understanding of the UART data transmitted on the STM32 end, an arrangement of four bytes was introduced: Start, Command, Data and Checksum. The `sendPacket()` function assembles the 4-byte frame and writes it atomically over `Serial2`. The checksum is computed as a bitwise XOR of the command and value bytes

```

21 void sendPacket(uint8_t cmd, uint8_t value)
22 {
23     uint8_t pkt[4] = { 0xFF, cmd, value, (uint8_t)(cmd ^ value) };
24     STM32_UART.write(pkt, 4);
25     Serial.printf("[UART TX] CMD=0x%02X VAL=%d\n", cmd, value);
26 }

```

Figure 18 ESP32 code snippet for sending the data packet to STM32

The purpose of each Byte is outlined below:

Byte 0 (0xFF): Fixed Start Byte. The STM32 waits for the start Byte to read the following bytes of the Packet. This provides synchronization despite loss of some bytes.

Byte 1 (cmd): Type of Command to be executed. Throttle control is 0x01, steering is 0x02, and emergency stop is 0xFF.

Byte 2 (value): Payload containing the information. The value 128 corresponds to neutral position for Throttle and Steering. Zero value indicates full left/reverse, while 255 indicates full right/forward.

Byte 3 (checksum): The Checksum value equals cmd XOR value. Any packet with mismatched values for the Checksum is rejected by the STM32.

COMMAND	CMD	VALUE	MEANING
THROTTLE FORWARD	0x01	128-255	Motor speed forward
THROTTLE REVERSE	0x01	0-127	Motor speed reverse
STEER RIGHT	0x02	128-255	Servo angle right
STEER LEFT	0x02	0-127	Servo angle left
EMERGENCY STOP	0xFF	0x00	Halt all motion

Table 2 The actual meaning of the bytes transmitted over the channel ESP32 -> STM32

4.3.3 Role of the ESP32

The ESP32 acts purely as a communication bridge. It has no knowledge of motors or servos — its only job is to receive joystick data from the Dabble app over BLE, translate the joystick axis values into the 4-byte UART packet format, and forward them to the STM32. This separation of concerns keeps each microcontroller's firmware focused and simple.

The firmware is written for the Arduino framework using the DabbleESP32 library. Three critical preprocessor directives must appear before any includes, as they configure which Dabble module is compiled in:

```
1  #define CUSTOM_SETTINGS
2  #define INCLUDE_GAMEPAD_MODULE
3  #include <DabbleESP32.h>
```

Figure 19 Libraries included

In setup(), the UART2 hardware peripheral is initialized on GPIO16/17 at 115200 baud to match the STM32 configuration, and BLE advertising is started under the device name "RC-Vehicle":

```

38
39 void setup()
40 {
41     Serial.begin(115200);
42     STM32_UART.begin(STM32_BAUD, SERIAL_8N1, STM32_RX_PIN, STM32_TX_PIN);
43     Dabble.begin("RC-Vehicle");
44     Serial.println("[BLE] Advertising as RC-Vehicle");
45 }

```

Figure 20 Hardware Peripheral Initialization

The main loop reads the Dabble gamepad joystick values and converts them into UART packets. A deadzone is applied around the center position to prevent drift when the joystick is barely touched. Change detection ensures packets are only sent when values actually change, reducing unnecessary traffic on the UART line:

```

46
47 void loop()
48 {
49     Dabble.processInput();
50
51     /* D-pad buttons */
52     if (GamePad.isUpPressed())
53     {
54         lastThrottle = 255;
55         lastSteering = 128;
56     }
57     else if (GamePad.isDownPressed())
58     {
59         lastThrottle = 0;
60         lastSteering = 128;
61     }
62     else if (GamePad.isLeftPressed())
63     {
64         lastThrottle = 128;
65         lastSteering = 0;
66     }
67     else if (GamePad.isRightPressed())
68     {
69         lastThrottle = 128;
70         lastSteering = 255;
71     }
72     else
73     {
74         /* Joystick mode */
75         float xRaw = GamePad.getXaxisData();
76         float yRaw = GamePad.getYaxisData();

```

Figure 21 Main Loop

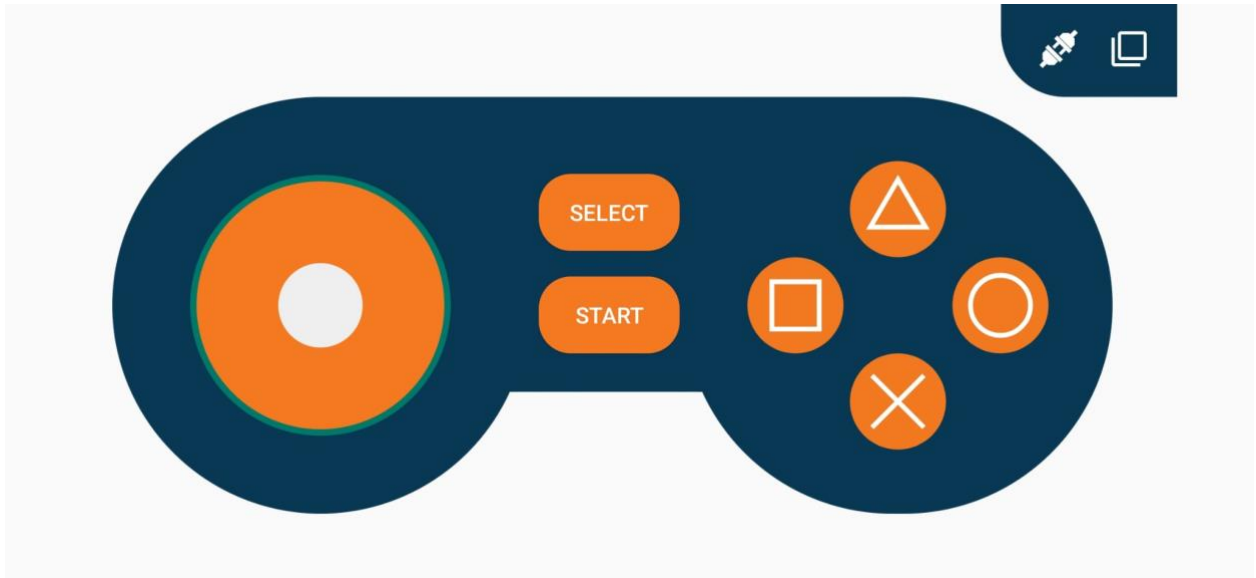


Figure 22 Dabble joystick gamepad

4.3.4 STM32- UART Reception and Vehicle Control

After CubeMX generates the peripheral initialization code, USART2 receive is enabled using interrupt-driven reception. HAL_UART_Receive_IT() registers a callback that fires each time a byte arrives, keeping the main loop free

```
}  
|  
| HAL_UART_Receive_IT(&huart2, &uart_rx_byte, 1);  
|  
}
```

Figure 23 UART Initialization

The interrupt callback implements a state machine that processes one byte at a time. It waits for the 0xFF start byte, collects CMD and VALUE, then validates the checksum before passing the command onwards. After each byte, the interrupt is re-armed to receive the next one:

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance != USART2)
        return;

    uint8_t byte = uart_rx_byte;

    if (packet_idx == 0)
    {
        if (byte == PACKET_START)
            packet_buf[packet_idx++] = byte;
    }
    else
    {
        packet_buf[packet_idx++] = byte;

        if (packet_idx >= PACKET_SIZE)
        {
            uint8_t cmd      = packet_buf[1];
            uint8_t value    = packet_buf[2];
            uint8_t checksum = packet_buf[3];

            if ((cmd ^ value) == checksum)
            {
                rc_cmd      = cmd;
                rc_value    = value;
                packet_ready = 1;
            }
            packet_idx = 0;
        }
    }
}

```

Figure 24 Packet Parser

The use of interrupt-driven UART reception on the STM32 is a key design decision. Rather than blocking the main loop by polling for data, the HAL interrupt callback processes bytes as they arrive, ensuring no bytes are missed. The checksum provides meaningful protection against line errors, a single flipped bit changes the checksum and the packet is discarded, preventing spurious commands from reaching the actuators.

4.4 Final Integrated Code

4.4.1 Overview

After testing each hardware component and ensuring they work, it was time to connect the components together. The first big change was for the DC motor logic. Before, DC motors were controlled by a timer, but with USART2 communication enabled, a timer was not needed anymore. Servo motor is still controlled by a timer so, our solution was configuring

the servo motor on Timer2 Channel 1, and the Motor1 and Motor2 pins that we would configure in code to control DC motor movement were set to GPIO_Output state. The pins are as follows, Motor1 IN1 corresponds to PB0, IN2 corresponds to PB10. For Motor2 IN1 and IN2 pins are PB4 and PB5. The final MX configuration was:

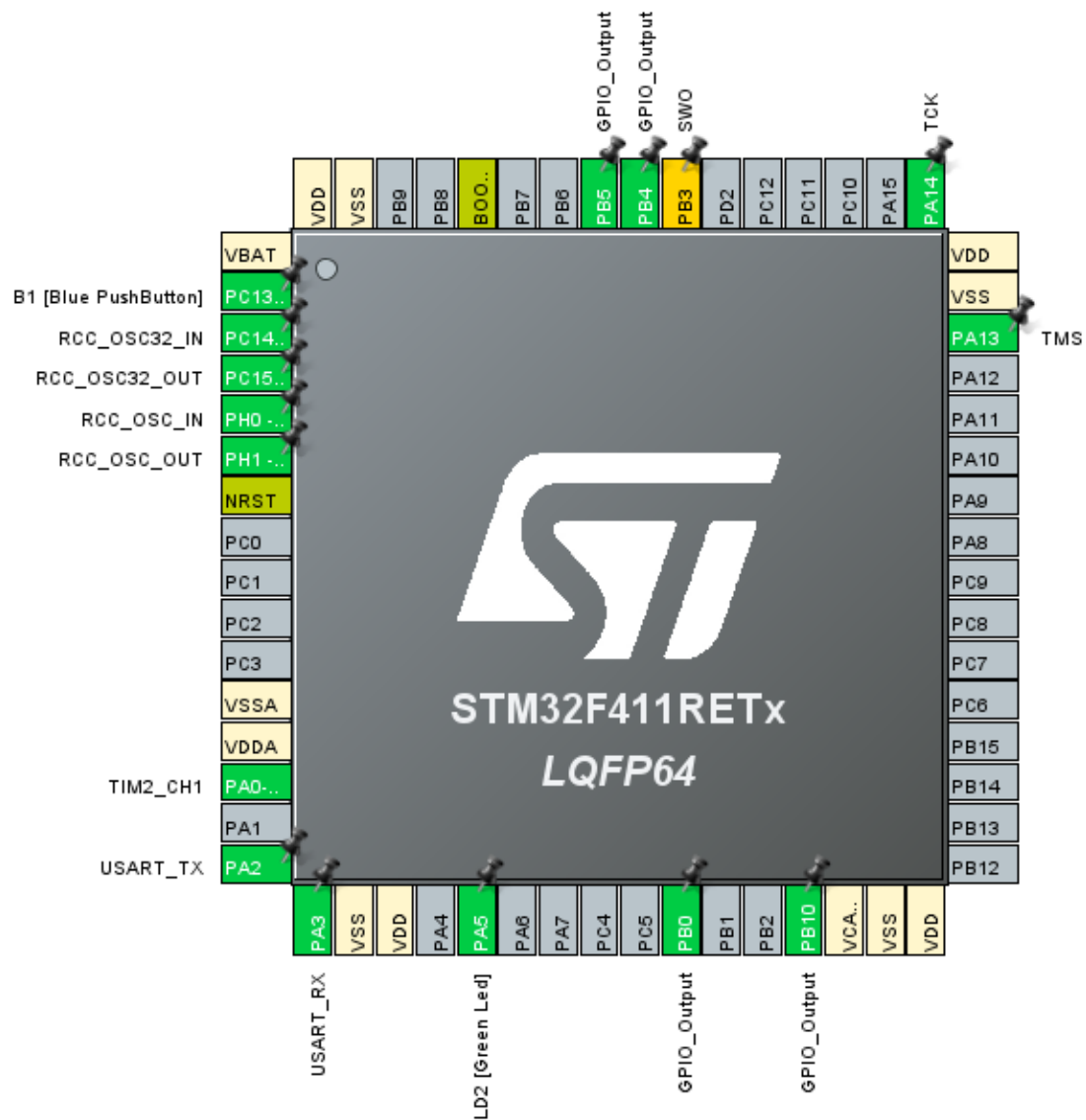


Figure 25 Final MX configuration

4.4.2 Limit definitions

Each component has their own limit definitions, pin definitions etc.

```

6
7 /* ===== SERVO ===== */
8 #define SERVO_MIN      1000
9 #define SERVO_CENTER    1500
10 #define SERVO_MAX       2000
11

```

Figure 26 Servo limits definition

These values define the PWM pulse width for the servo motor. The servo is controlled using a 50 Hz PWM signal, where the pulse width decides the angle.

1000 means the servo turns to one side, 1500 is the center position, and 2000 turns it to the opposite side. This makes the steering easier to control because the rest of the code can use these named values instead of random numbers.

```

1
2 /* ===== MOTOR PINS ===== */
3 #define IN1_PORT GPIOB
4 #define IN1_PIN  GPIO_PIN_0
5 #define IN2_PORT GPIOB
6 #define IN2_PIN  GPIO_PIN_1
7 #define IN3_PORT GPIOB
8 #define IN3_PIN  GPIO_PIN_4
9 #define IN4_PORT GPIOB
10 #define IN4_PIN  GPIO_PIN_5

```

Figure 27 Motor driver input pins

These definitions connect the STM32 pins to the motor driver input pins. The L298N motor driver uses these four signals to control the direction of the two DC motors. By defining them at the top, the motor functions become much cleaner and easier to understand.

```

/* ===== UART PROTOCOL ===== */
#define PACKET_SIZE      4
#define PACKET_START     0xFF
#define CMD_THROTTLE     0x01
#define CMD_STEERING     0x02
#define CMD_STOP         0xFE
#define THROTTLE_DEADZONE 10
#define COMM_TIMEOUT_MS  500

```

Figure 28 USART2 definitions

As mentioned above, each packet has 4 bytes including: start byte, command, value and checksum. These are defined here and their purpose is reusability in code.

4.4.3 Motor functions

```

56 void motors_stop(void)
57 {
58     HAL_GPIO_WritePin(IN1_PORT, IN1_PIN, GPIO_PIN_RESET);
59     HAL_GPIO_WritePin(IN2_PORT, IN2_PIN, GPIO_PIN_RESET);
60     HAL_GPIO_WritePin(IN3_PORT, IN3_PIN, GPIO_PIN_RESET);
61     HAL_GPIO_WritePin(IN4_PORT, IN4_PIN, GPIO_PIN_RESET);
62 }

```

Figure 29 Function for stopping motors

This function stops both DC motors by setting all motor driver inputs to LOW. It is used when the joystick is centered, when communication is lost, or when an emergency stop command is received.

```

64 void motors_forward(void)
65 {
66     HAL_GPIO_WritePin(IN1_PORT, IN1_PIN, GPIO_PIN_SET);
67     HAL_GPIO_WritePin(IN2_PORT, IN2_PIN, GPIO_PIN_RESET);
68     HAL_GPIO_WritePin(IN3_PORT, IN3_PIN, GPIO_PIN_SET);
69     HAL_GPIO_WritePin(IN4_PORT, IN4_PIN, GPIO_PIN_RESET);
70 }
71

```

Figure 30 Function for forward rotation of motors

This function makes both motors rotate forward. For each motor, one input pin is HIGH and the other is LOW. This tells the motor driver which direction current should flow through the motor.

```
72 void motors_reverse(void)
73 {
74     HAL_GPIO_WritePin(IN1_PORT, IN1_PIN, GPIO_PIN_RESET);
75     HAL_GPIO_WritePin(IN2_PORT, IN2_PIN, GPIO_PIN_SET);
76     HAL_GPIO_WritePin(IN3_PORT, IN3_PIN, GPIO_PIN_RESET);
77     HAL_GPIO_WritePin(IN4_PORT, IN4_PIN, GPIO_PIN_SET);
78 }
79
```

Figure 31 Function for backward rotation of motors

This does the opposite of the forward function. The input pin states are reversed, so the motors rotate backward.

4.4.4 Steering functions

```
82 void apply_throttle(uint8_t value)
83 {
84     if (value > (128 + THROTTLE_DEADZONE))
85         motors_forward();
86     else if (value < (128 - THROTTLE_DEADZONE))
87         motors_reverse();
88     else
89         motors_stop();
90 }
91
```

Figure 32 Applying throttle from joystick value

This function converts received throttle value into movement. Firstly, 128 is treated as a center value, if value received from joystick movement is greater than 128 the value results in forward movement, if the value is less than 128 that is for backwards movement. The deadzone is important because joystick values are not always perfectly stable. Without it, the motors could move even when the user is not intentionally pushing the joystick.

```

94=void apply_steering(uint8_t value)
95 {
96     uint32_t pulse;
97
98     if (value < 118)
99         pulse = SERVO_MIN + ((uint32_t)value * (SERVO_CENTER - SERVO_MIN)) / 117;
100     else if (value > 138)
101         pulse = SERVO_CENTER + ((uint32_t)(value - 138) * (SERVO_MAX - SERVO_CENTER)) / 117;
102     else
103         pulse = SERVO_CENTER;
104
105     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, pulse);
106 }
107

```

Figure 33 Steering control

This code controls the servo motor for steering. The received value is converted into a PWM pulse width. If the value is below 118, the servo moves to one side, if the value is above 138 it moves toward the other. Otherwise, servo stays centered. The last line of this function is most important since it changes servo position without stopping the movement, so servo position can change while in motion.

```

10=void emergency_stop(void)
11 {
12     motors_stop();
13     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, SERVO_CENTER);
14 }
15

```

Figure 34 Emergency stop

This function immediately stops the vehicle and centers the steering servo. It is used when a stop command is received or when communication is lost.

This is an important safety feature because the vehicle should not keep moving if the controller disconnects or if the ESP32 stops sending data.

4.4.5 Processing and RC initialization

```
8 void process_packet(uint8_t cmd, uint8_t value)
9 {
10     last_packet_tick = HAL_GetTick();
11     switch (cmd)
12     {
13         case CMD_THROTTLE: apply_throttle(value); break;
14         case CMD_STEERING: apply_steering(value); break;
15         case CMD_STOP:     emergency_stop();      break;
16         default: break;
17     }
18 }
19
```

Figure 35 Packet processing

This function is the main logic for receiving signals. Firstly, the signal needs to be valid, if it is there are three possible commands: throttle which calls the `apply_throttle` function from before. Steering which sends the value to the servo motor, and stop which calls the `emergency_stop` function. The first line in this function updates the time of the last received command.

```
31
32 void RC_Init(void)
33 {
34     HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
35
36     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, SERVO_MIN);
37     HAL_Delay(1000);
38     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, SERVO_MAX);
39     HAL_Delay(1000);
40     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_3, SERVO_CENTER);
41     HAL_Delay(500);
42
43     motors_stop();
44     last_packet_tick = HAL_GetTick();
45     HAL_UART_Receive_IT(&huart2, &uart_rx_byte, 1);
46 }
47
```

Figure 36 RC initialization

This function prepares the vehicle before the main loop starts.

First, it starts PWM generation on TIM2 Channel 3, which is needed for the servo. Then it moves the servo to minimum, maximum, and center positions. This acts like a startup test to confirm that the servo is receiving the PWM signal correctly.

After that, the motors are stopped, the communication timer is reset, and UART interrupt reception is started using the last line. This means that STM32 receives UART inputs as interrupts rather than constantly checking for the UART data inside the loop.

```

.50 void RC_Process(void)
.51 {
.52     if (packet_ready)
.53     {
.54         packet_ready = 0;
.55         process_packet(rc_cmd, rc_value);
.56     }
.57
.58     if ((HAL_GetTick() - last_packet_tick) > COMM_TIMEOUT_MS)
.59     {
.60         emergency_stop();
.61         last_packet_tick = HAL_GetTick();
.62     }
.63 }
.64

```

Figure 37 Main RC processing loop

This function runs repeatedly while the vehicle is on. Firstly, it checks if a valid packet is received, if yes packet_ready becomes 1 until the command is finished and a new packet is received. The second part checks communication. If no valid command is sent for a certain amount of time the vehicle automatically stops making it safer, since no random movement should be expected from the vehicle unless a command is sent and prevents movement if connection is lost.

```

5
1= int main(void)
5 {
5     HAL_Init();
7     SystemClock_Config();
3     GPIO_Init();
9     USART2_Init();
0     TIM2_PWM_Init();
1     RC_Init();
2
3     while (1)
4     {
5         RC_Process();
5     }
7 }
3

```

Figure 38 Main function

The main function initializes the STM32, configures the GPIO pins, UART communication, and PWM timer, then starts the custom RC vehicle logic.

Inside the infinite loop, RC_Process() continuously checks whether a new packet was received or whether the communication timeout has occurred.

5. Incremental Results and Final Results

5.1 Overview

This section presents the progress of the RC vehicle project in separate testing stages. Instead of building and testing the full system at once, each main component was tested individually first. The DC motors, servo motor, UART communication, and final combined vehicle control were tested step by step. This approach made it easier to identify errors, fix connection or code problems, and confirm that each part of the system worked correctly before moving to the next stage. The incremental testing also helped show how the project developed from simple motor movement to a complete RC vehicle controlled through commands sent from the ESP32 to the STM32 microcontroller.

5.2 DC motors

The DC motors were tested through the L298N motor driver to confirm forward and reverse movement.

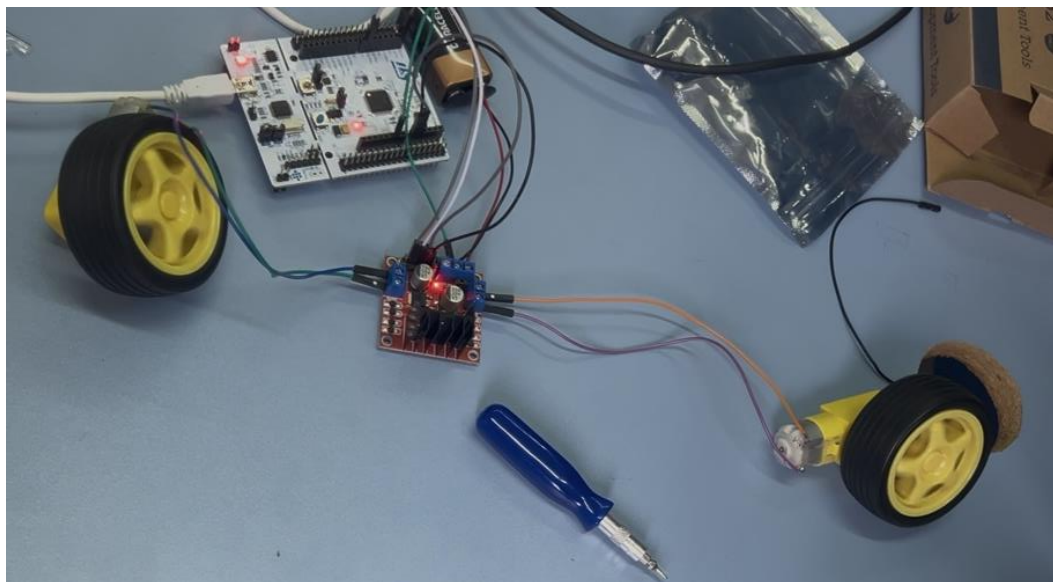


Figure 39 DC motors connected to motor driver

5.3 Servo motor

The servo motor was tested using PWM to check left, right, and center steering positions.

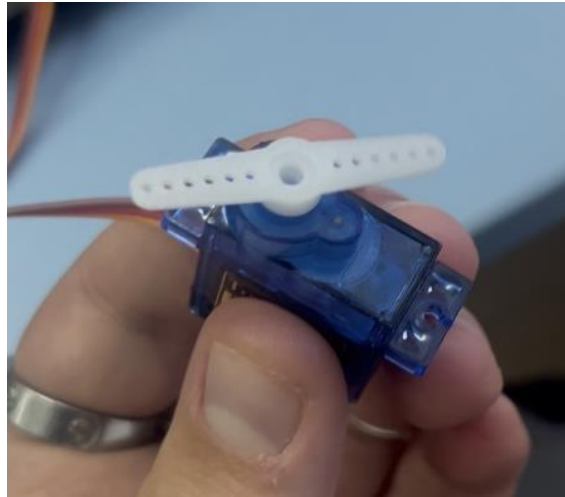


Figure 40 Servo Motor rotation

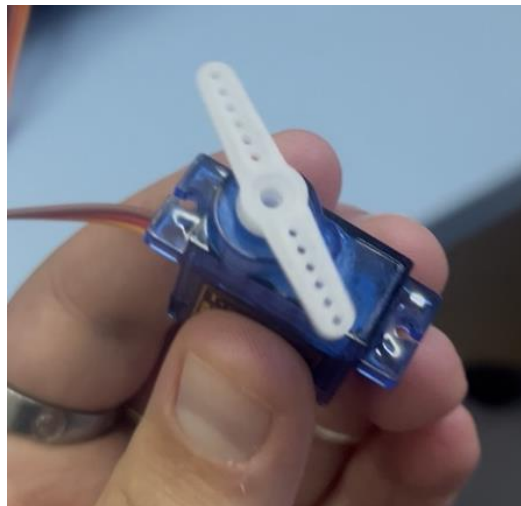


Figure 41 Servo motor rotation

5.4 Setup using Buck converter

The initial completed setup, and envisioned final product included a step-down regulator, also known as a Buck converter. This is ideally what is used to step-down the circuit voltage from 9V to the needed amount. In our case, we would need 3 rails of voltage: 9V for the motors, 3.3V for STM32 and 5V for ESP32. The other issue was current, while the step-down regulator would lower the voltage it would also lower the current significantly. This issue, in theory, could be resolved by adding a load, a capacitor, which we tried as well. But the capacitance would have to be significantly high for the needed current and to slow capacitor charging .

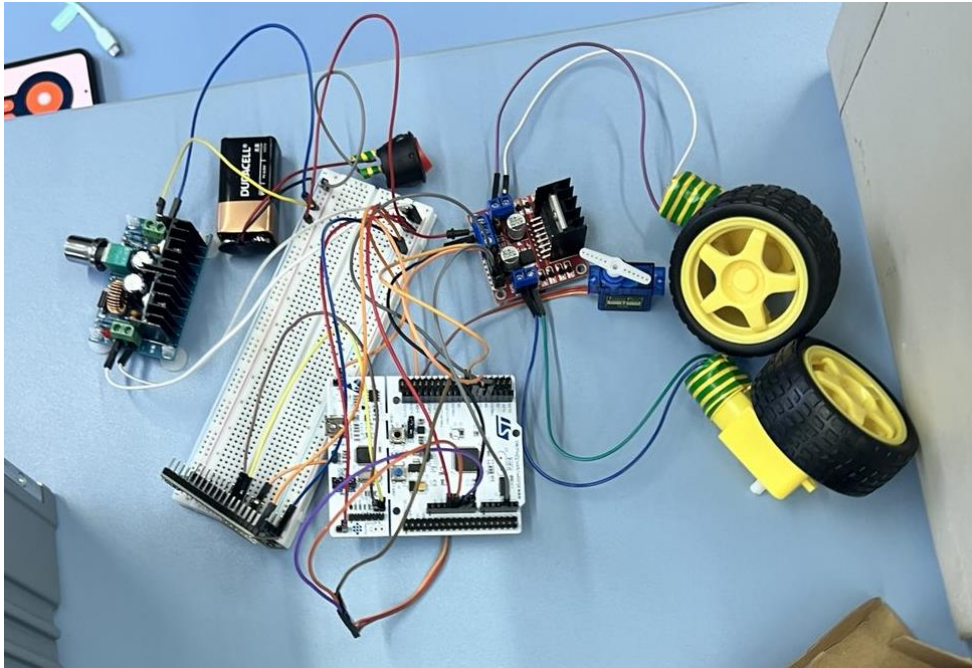


Figure 42 Connected components including Buck converter

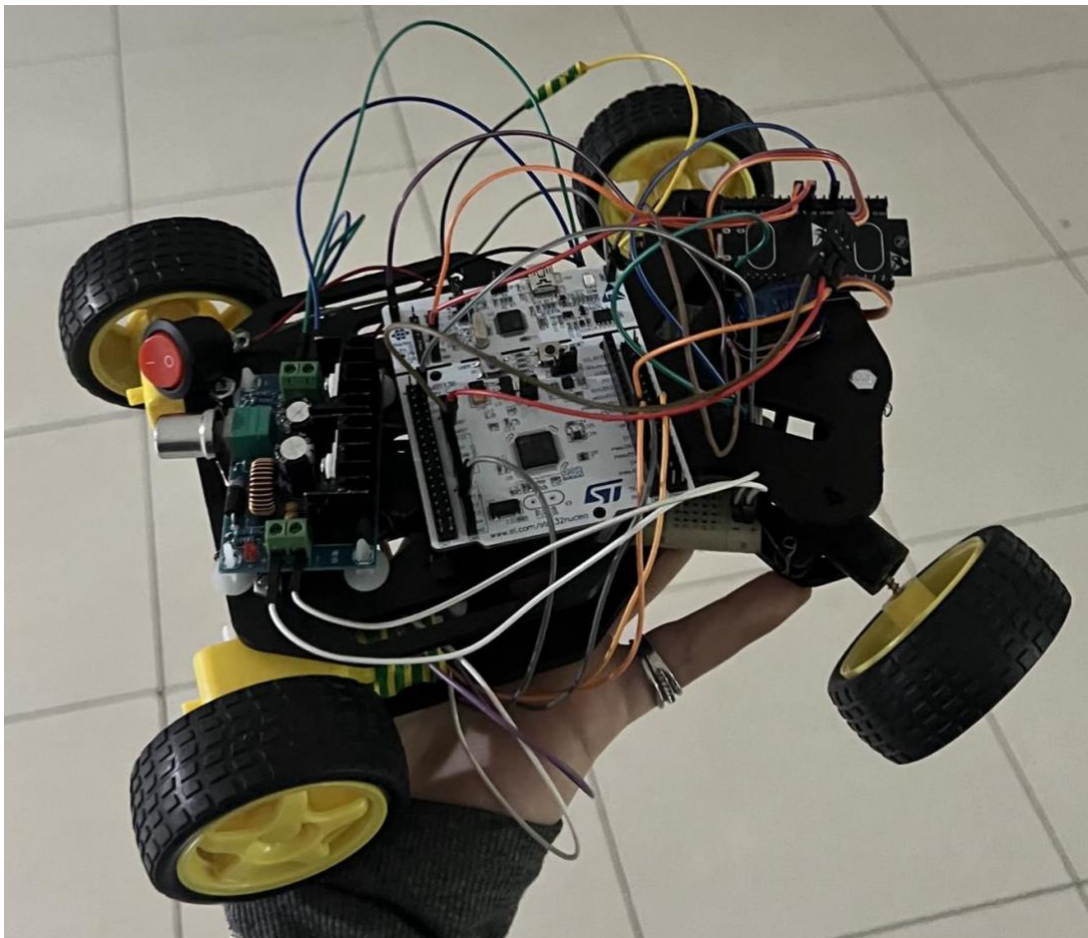


Figure 43 Components placed on chassis

5.5 Final setup

After individual testing, all components were connected together and tested as the final RC vehicle system. Our final solution for the need of constant current for the microcontrollers was using a powerbank.

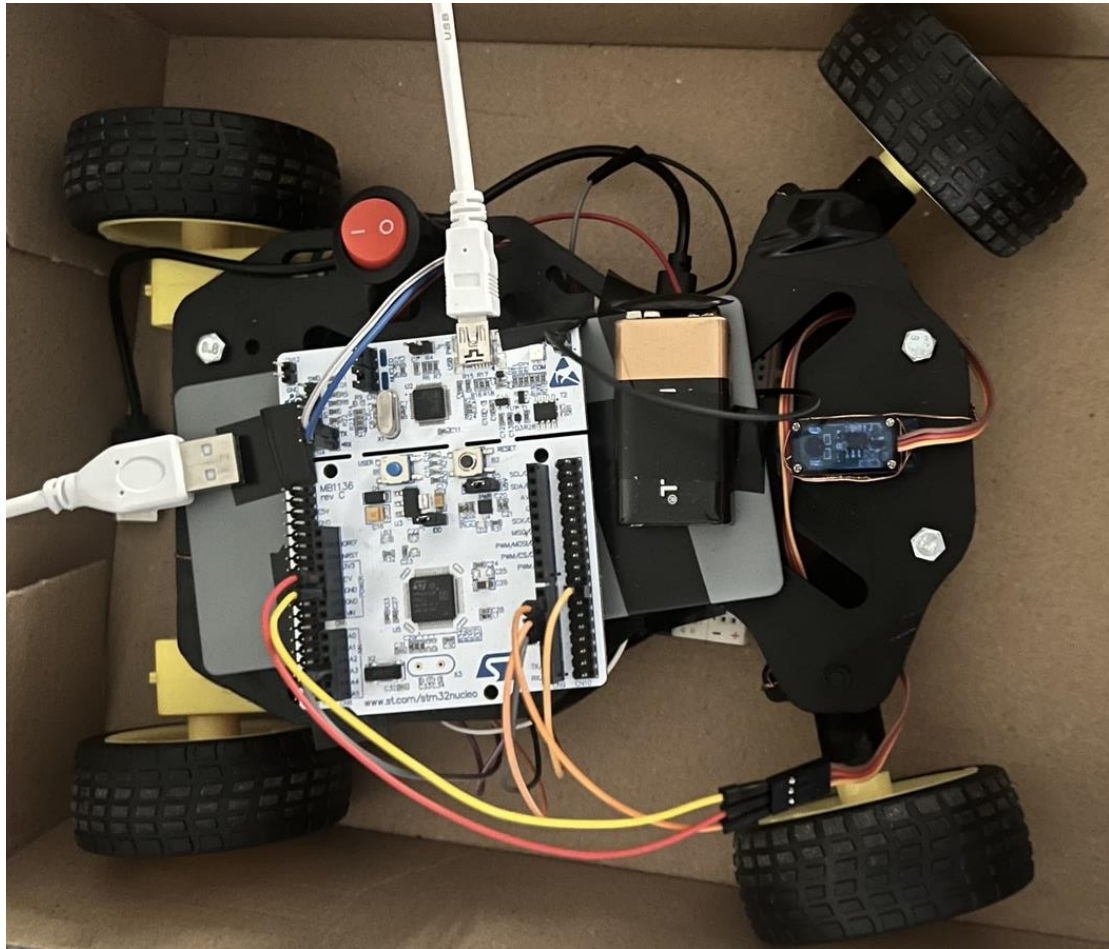


Figure 44 Final RC vehicle setup

6. Challenges and Troubleshooting

During the development of the RC vehicle project, we spent approximately 85 hours in the laboratory. Throughout this period, several challenges occurred, ranging from equipment availability and hardware faults to soldering, wiring, code conflicts, and power supply limitations. The following sections outline the main problems we encountered, their impact on the project, and the solutions applied.

6.1 Challenges with the L298N Motor Driver

The L298N motor driver caused issues in two main ways. The first and most significant problem was finding a functional driver. The first two drivers we tested had hardware faults or missing components, making them unusable for the project. The third driver was tested successfully and was used in the final setup.

The second issue appeared while testing the connection between the L298N motor driver and the two DC motors. The rightmost pins on the motor driver were missing the connector cap needed to complete the connection, which caused one of the DC motors to stop working. Initially, we used a resistor wire coiled around the pins as a temporary fix. Later, we replaced it with a proper connector cap borrowed from one of the STM32 pin headers, which created a more stable connection.

6.2 Challenges in Soldering and Wiring

Soldering and wiring issues mainly occurred due to the loose connections on the DC motors. During the first few soldering attempts, the absence of flux paste made the process more difficult because the solder did not properly remain on the soldering iron. This caused weak connections between the cut wires and the DC motor terminals. After finding flux paste in the Control Laboratory, the soldering process became cleaner, and the connections became stronger and more reliable.

A smaller wiring issue occurred when tools such as a knife, pliers, and a lighter were forgotten. Since no suitable substitutes were available in the laboratory at that time, this temporarily slowed down the wiring process.

6.3 Challenges with the Servo Motor

The main challenges related to the servo motor were caused by code configuration issues, especially conflicts involving timers and UART communication. A smaller issue was finding a functional servo motor, but this was quickly solved by replacing it with another servo from the laboratory.

6.4 Challenges Caused by Timer and UART Conflicts

The most complex issue occurred when combining the code for the servo motor with the code for the two DC motors. The first conflict happened because both the DC motors and the servo motor were configured to use Timer 2. This was solved by changing the DC motor control to GPIO-based control instead of timer-based PWM control.

However, another conflict appeared between the servo motor configuration and UART communication. At first, UART was disabled to test whether it was causing the issue, but this was not a suitable final solution because UART was required for communication between the STM32 and the ESP32. The conflict was eventually resolved by moving the servo motor control to Timer 3, allowing UART communication to remain active while the servo motor operated correctly.

6.5 Challenge Related to Current Supply

The final challenge was related to the current supply. Initially, one 9V battery was used to power all components, including the L298N motor driver, STM32 board, and ESP32 module. A buck converter was used to regulate the voltage to 3.3V for the STM32 and 5V for the ESP32. Although the voltage levels were correct, the RC car still did not function as intended.

After testing, it became clear that the issue was caused by insufficient current. The L298N motor driver and the two DC motors drew a significant amount of current from the 9V battery, which affected the stability of the other components. To solve this problem, a power bank was used to provide a more reliable current supply for the STM32 and ESP32, ensuring that both modules could function properly during operation.

7. Conclusion

In conclusion, an RC vehicle project proved to be challenging, but managed to connect both theoretical knowledge obtained during the course including PWM, timer and clock configuration with real hardware components and materials. It is one thing to talk about a certain topic, and to write the code for a hardware component with that knowledge in mind. We believe that we learned a lot through this project and have nothing but inspiration for projects further down the line.



Figure 45 RC vehicle project at the Makers Faire 2026

References

- [1] STMicroelectronics. (2023). STM32F411xC/E advanced Arm-based 32-bit MCUs reference manual (RM0383).
https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STEMpedia. (n.d.). DabbleESP32 [Software library]. GitHub.
<https://github.com/STEMpedia/DabbleESP32>
- [3] CreatorUsername. (n.d.). *Title of the 3D model/design*. Thingiverse.
<https://www.thingiverse.com/thing:4772729>
- [4] Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai/>